8 de abr. de 2026

C# "Avançado do Básico"

Convidados Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider Gabriel Severo Gabriel kohlrausch Ruan Rodrigues aline.bianchini@t-systems.com ana.santos@t-systems.com analista.kuehn@outlook.com anderson.hilario@t-systems.com anderson.perim@t-systems.com andersongabriel_maria@hotmail.com anselmogabriel421@gmail.com bmf.pedro@gmail.com bruna.nery@t-systems.com bruno.destro@t-systems.com caique.rodrigues@t-systems.com d.silva@t-systems.com danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com eduardo.kubota@t-systems.com eudy.amparo@t-systems.com fabio.marinelli@t-systems.com farley.t.i@hotmail.com felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com hewerton.souza@t-systems.com i.parmigiani@t-systems.com igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com l.regioli@outlook.com leandro.piqueira@t-systems.com lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com marcelohcordeiro@gmail.com marcio@almob.com.br marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com renan-silva@t-systems.com renan.baptista.cabral@gmail.com rodrigo_7sch@hotmail.com tatidornel@gmail.com thiago.fernandes@t-systems.com victor.prospero@t-systems.com victor.santos@t-systems.com victor.souza@t-systems.com vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com wesley.boccaletto@t-systems.com wsilva@t-systems.com fabiano.felgas@gmail.com

Anexos C# "Avançado do Básico"

Registros da reunião Gravação

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Resumo

O curso de otimização de código e profiling detalhou a imutabilidade de strings, o impacto da alocação de memória em alta escala via *Value Type* e *Reference Type*, com discussões sobre o funcionamento interno do *Garbage Collector* e a performance de coleções.

Otimização e Imutabilidade de Strings
A discussão sobre otimização de código mostrou que `Replace` em um loop aloca 3 vezes mais memória que `String.Format` devido à imutabilidade da string. Esta otimização em um sistema de alta escala resolveu um problema de falta de memória no Kubernetes POD, demonstrando o impacto crucial da alocação de memória.

Gerenciamento de Memória e GC
Os cinco pilares de gerenciamento de memória foram introduzidos, detalhando que *Value Types* são alocados na Stack para rápida desalocação, e *Reference Types* são alocados na Heap, gerenciada pelo *Garbage Collector* (GC). O GC limpa a Heap em três gerações, e a alocação excessiva pode suspender a aplicação e levar à fragmentação na *Large Object Heap*.

Tipos de Dados e Estruturas
Apresentou-se o `record` como açúcar sintático para uma classe imutável que compara por valor, sendo ideal para DTOs. O *benchmark* de coleções demonstrou que `Hashset` e `Dictionary` têm tempo de acesso constante, sendo superiores às listas para buscas rápidas, pois utilizam *hashtable*.

Detalhes

- **Introdução e Agenda do Curso**: Bruno Joaquim iniciou a primeira aula oficial do curso, com o objetivo de aprofundar-se em conceitos teóricos, especialmente aqueles relacionados ao profiling. Eles mencionaram que o curso foi estruturado para maximizar a amplitude técnica dos participantes,

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

aprofundando-se em tópicos cruciais e revisando outros de forma mais superficial, garantindo que os participantes desenvolvam uma linha de estudo para necessidades futuras. O instrutor convidou os participantes a interagirem e tirarem dúvidas a qualquer momento, mesmo que as questões não estivessem diretamente relacionadas ao material apresentado.

- **Análise de Otimização de Código SQL (String Replacement vs. String Format)**: Bruno Joaquim apresentou um exemplo real de produção onde duas abordagens diferentes resolviam o mesmo problema de construir um SQL dinâmico baseado em mensagens de um Kafka. A primeira abordagem usava `Replace` em um loop, e a segunda usava `String.Format` para a manipulação da string. O objetivo era discutir a diferença entre as duas abordagens, além da sintaxe, focando na alocação de memória.
- **Discussão sobre Alocação de Memória em Strings (Imutabilidade)**: Lucas Pereira e Felipe Neves notaram que a primeira abordagem, usando `Replace` dentro de um loop, alocaria uma nova string na memória para cada condição, pois o método `Replace` constrói uma nova string. Bruno Joaquim confirmou que strings são imutáveis e que o `Replace` reatribui um novo valor, realocando o espaço em memória múltiplas vezes se houver muitas condições. A segunda abordagem, usando `String.Format`, faz apenas uma alocação adicional, semelhante ao que o `StringBuilder` faz internamente.
- **Resultados de Benchmark e Impacto em Sistemas de Alta Escala**: Um benchmark executado com a biblioteca `Benchmark.net` mostrou que a abordagem `Replace` alocou aproximadamente três vezes mais memória do que o `String.Format`. Embora a diferença de tempo de execução (nanossegundos) seja insignificante, a alocação de memória é crucial em sistemas de alta escala. O ajuste desse código em um projeto com 15 a 20 milhões de integrações semanais resolveu um problema de queda do POD Kubernetes, que estava esgotando a memória configurada.
- **Apresentação dos Cinco Pilares de Gerenciamento de Memória**: Bruno Joaquim introduziu os cinco pilares essenciais a serem dominados no gerenciamento de memória em .NET. Os tópicos a serem abordados incluem: *Value Type* e *Reference Type*, *Heap* e *Stack* (tipos de memória), *Boxing* e *Unboxing*, *Class* e *Record*, e o *Garbage Collector* (GC).
- **Conceito de Value Type (Tipos de Valor)**: Os *Value Types* são tipos que carregam um valor e sempre representam o valor atribuído, incluindo tipos primitivos (int, double, bool) e *structs*. Ao passar um *Value Type* como parâmetro, uma cópia do valor é utilizada, não a referência. Se houver a necessidade de passar o valor original, deve-se usar a palavra-chave `ref`.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Comparação de Value Types e Structs**: Na comparação entre dois *Value Types*, o que é comparado é sempre o valor e não a referência. Uma `struct` é um *Value Type* e a comparação entre duas structs compara o conteúdo campo a campo, a menos que o método `Equals` seja sobrescrito.
- ****Uso da Palavra-Chave `ref` e Implicações na Cópia de Valores****: O uso de `ref` permite a passagem de uma referência ao valor original, mas a atribuição desse valor a uma nova variável dentro do método ainda gera uma cópia. Eles concluíram que usar `ref` para otimizar a cópia de um *Value Type* não é benéfico para o desempenho, pois a desalocação de *Value Types* na Stack é rápida, e aumenta o risco de modificar o valor original, o que é má prática.
- **Relação entre Value Types e Alocação de Memória (Stack)**: Farley Souza questionou se a cópia de *Value Types* é custosa, e Bruno Joaquim esclareceu que não, pois *Value Types* geralmente são pequenos e alocados na Stack. A Stack desaloca essas variáveis rapidamente ao final do método, sem envolver o Garbage Collector.
- **Uso de Structs e Riscos de Stack Overflow**: Gustavo Ferraz Fontes levantou uma questão sobre o uso indiscriminado da Stack (por exemplo, com muitas *structs*) e o risco de *Stack Overflow*. Bruno Joaquim explicou que a Stack tem um tamanho limitado (4MB em sistemas 64 bits) e o uso irresponsável poderia levar a um erro não recuperável (*Stack Overflow*). Ele também mencionou a otimização no .NET 10, que aloca listas pequenas na Stack para evitar alocações desnecessárias na Heap.
- **Restrições de Structs e Alocação em Heap (Reference Type)**: É importante evitar colocar *Reference Types* dentro de *Structs*. Se uma *struct* estiver dentro de uma classe, toda a estrutura, incluindo a *struct*, será alocada na Heap, anulando o benefício de alocação na Stack. Eles recomendaram o uso de *structs* para objetos simples (como DTOs anêmicos) para evitar alocação na Heap.
- **Conceito de Reference Type (Tipos de Referência)**: *Reference Types* são ponteiros para uma região da memória, e passar um *Reference Type* como parâmetro resulta na passagem da referência para o objeto original. Eles são sempre alocados na Heap e o `Equals` padrão compara a referência, não o valor (a menos que seja sobrescrito).
- **O Comportamento Único da String**: A `string` é um *Reference Type* (alocada na Heap) mas, para fins de comparação, ela se comporta como um *Value Type*. Isso significa que a comparação (`==` ou `Equals`) verifica o valor

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

intrínseco da string e não sua referência de memória, devido a uma sobrecarga (overload) do operador `==` no código fonte do .NET.

- **Memória Stack e Heap em Detalhe**: A Stack armazena *Value Types* e referências para *Reference Types* e é rápida e previsível, removendo as variáveis imediatamente após a finalização do método. A Stack tem um tamanho limitado (4MB para 64 bits) e cada thread tem a sua própria Stack. A Heap armazena os *Reference Types*.
- **Stack Allocation e Value Types (Revisão)**: Bruno Joaquim demonstrou com um exemplo a rápida desalocação de *Value Types* na Stack após a execução de um método, reforçando que o uso de `ref` para evitar a cópia não traz ganhos de performance e pode ser arriscado. Eles concluíram que a alocação pontual de *Value Types* não esgota a Stack.
- **Tipos de Valor e Tipos de Referência**: Fabiano Felgas perguntou sobre a duplicação do espaço na memória ao passar um objeto como parâmetro, questionando se a passagem por referência utiliza apenas o espaço alocado. Bruno Joaquim explicou que a duplicação ocorre apenas com "value type" quando não se usa o *keyword* `ref`. O uso de `ref` só faz sentido com "value type", que não vai para a *Heap* (Pilha). Tipos de referência (*reference type*) são sempre por referência, passando um ponteiro de memória por parâmetro e, portanto, não duplicam a informação.
- ****Alocação na *Heap* (Pilha) e o *Garbage Collector****: Bruno Joaquim explicou que qualquer instância de *reference type* é armazenada na *Heap* e precisa ser coletada pelo *Garbage Collector* (GC). A *Heap* é referenciada como *managed* porque é gerenciada pelo GC, e ela não tem um tamanho fixo, podendo solicitar mais memória ao sistema operacional se necessário, resultando em uma *Heap* segmentada. A limpeza na *Heap* é sempre feita pelo GC, e todas as *threads* compartilham a mesma *Heap*.
- ****Limpeza e Funcionamento Conjunto de *Stack* e *Heap****: O *runtime* solicita uma quantidade determinada de memória ao sistema operacional para o processo, e o GC é responsável por limpar objetos que não têm mais referência. Objetos na *Heap* podem sobreviver além do escopo do método que os criou, e o GC determina quais instâncias devem ser limpas.
- **Boxing e Unboxing**: Marcelo Henrique e Lucas Pereira foram questionados sobre *boxing* e *unboxing*. Bruno Joaquim definiu *boxing* como a conversão de um *value type* para um *reference type* (ex: `int` para `object`), movendo a informação da *Stack* para a *Heap*. O *unboxing* é o processo inverso, sendo o *boxing* mais custoso por acarretar alocação na *Heap*.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Impacto do Boxing na Concatenação de *Strings***: Ao usar `string.Concat` e passar um *value type* (como um inteiro) para um parâmetro que espera um `object`, a operação exige um *boxing*. Essa conversão aloca memória adicional na *Heap*, o que torna essa operação custosa.
- **Interface `IEquatable` para Evitar *Boxing***: Bruno Joaquim recomendou usar a interface `IEquatable` para tipos *struct* para evitar *boxing* desnecessário em comparações. Sem o `IEquatable`, a comparação de dois *structs* exigiria a transformação deles em `object`, resultando em *boxing* e alocação desnecessária na memória. Ao implementar `IEquatable`, a sobrecarga de comparação é usada, evitando a alocação na *Heap* e o *boxing*.
- **Comparação de Desempenho (*Benchmark*) com e sem *Boxing***: Um *benchmark* mostrou que a comparação de pontos sem *boxing* resulta em zero alocação. Em contraste, a mesma operação com *boxing* aloca 24 bytes para cada comparação, destacando o risco de estourar a memória em cenários com milhões de comparações por segundo.
- **Classes versus *Structs* na Modelagem**: Farley Souza levantou a preocupação de que o uso de classes deve ser mais bem avaliado, pois geralmente pensam em classes automaticamente para modelar. Bruno Joaquim concordou, explicando que classes são mais usadas para modelar entidades ricas de domínio com comportamentos e identidades customizadas. Para objetos de curta duração, DTOs ou informações com pouco valor sob a perspectiva de negócio, o ideal é usar *struct* ou *record* *struct*.
- **Comportamento do *Garbage Collector* (GC) e Segmentação da *Heap***: A *Heap* pode ficar fragmentada se a alocação for mais rápida que a limpeza do GC, levando o sistema a pedir mais memória, o que pode causar o erro *Out of Memory* se não houver mais memória disponível. O GC evita a concorrência de acesso à memória suspendendo a aplicação durante sua execução.
- **Etapas do *Garbage Collector***: O GC consiste em quatro etapas: suspensão da aplicação, marcação de objetos com referência (principalmente na *Stack*), desalocação de objetos não marcados e, finalmente, compactação para evitar fragmentação da *Heap*.
- **Gerações do *Garbage Collector***: O *managed Heap* é organizado em três gerações (Gen 0, Gen 1, e Gen 2). A Gen 0 é a área inicial de alocação de novos objetos (classes, listas, arrays), onde o GC atua com mais frequência

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

para liberar objetos temporários. Objetos que sobrevivem à limpeza da Gen 0 são promovidos para a Gen 1, que atua como ponte para a Gen 2. A Gen 2 é para objetos de longa duração (como um *singleton* em ASP.NET Core) que persistem na memória por muito tempo.

- **Impacto da Injeção de Dependência na Gerência de Memória**: Farley Souza perguntou sobre a relação das gerações do GC com injeção de dependência. Bruno Joaquim explicou que um *singleton* é promovido à Gen 2 porque persiste na memória, resistindo às limpezas do GC, e não porque o *runtime* o direciona diretamente para lá. Objetos com escopo (*scoped*) tendem a viver na Gen 0 e são descartados após o fim da requisição.
- **Performance da *Stack* vs. *Heap***: Karine Trevisan resumiu que a *Stack* é muito mais rápida e não utiliza o GC, enquanto a *Heap* depende do GC. Bruno Joaquim esclareceu que o problema de performance da *Heap* não é a alocação em si, mas a suspensão da aplicação que o GC precisa fazer repetidamente quando há alocação irresponsável e frequente.
- ***Structs* em Classes e o *Heap***: Se um *struct* estiver dentro de uma classe, e essa classe for instanciada, o *struct* será alocado na *Heap* junto com o restante da classe. No entanto, se o *struct* for usado isoladamente, ele será alocado temporariamente na *Stack*.
- **O *Large Object Heap* (LOH)**: O LOH é uma região da *Heap* exclusiva para arquivos ou objetos que excedem 85.000 bytes. Objetos no LOH, como um array de `int`, são tratados como *reference types* e alocados lá. O *Full GC* (limpeza em todas as gerações) também limpa o LOH.
- **Consequências da Alocação Excessiva no LOH**: Alocar frequentemente no LOH pode levar à execução mais frequente do *Full GC*, aumentando o tempo que a aplicação fica suspensa. Como a compactação não ocorre no LOH (devido ao custo), pode ocorrer fragmentação. A paginação de resultados de bancos de dados é recomendada para evitar o LOH.
- **Ferramentas para Profiling de Memória**: Fabiano Felgas perguntou sobre ferramentas para identificar o uso da memória *Heap*. Bruno Joaquim recomendou o DotMemory, que mostra o uso de memória na Gen 0, Gen 1 e Gen 2. O crescimento constante das gerações 1 e 2 é um forte indício de que a aplicação está criando objetos de longa duração que o GC não consegue limpar na Gen 0.
- **Melhores Práticas com *Strings***: Bruno Joaquim destacou que a *string* é imutável e sua modificação constante aloca mais memória. Recomenda-se o uso de `StringBuilder` ou `string.Format` e `ReadOnlySpan` para evitar

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

alocações desnecessárias, especialmente ao concatenar listas ou acessar partes de uma *string*.

- **Funcionamento Interno do *StringBuilder***: O `StringBuilder` é implementado como uma lista encadeada de arrays de *char*. Internamente, o tamanho máximo desses arrays é de 8.000 bytes, justamente para evitar a alocação no LOH (que começa em 85.000 bytes). O `ToString()` final do `StringBuilder` concatena todos esses *buckets* em uma única *string*.
- **Uso de *ReadOnlySpan* para Otimização de Memória**: *ReadOnlySpan* permite acessar uma seção de uma *string* (ou outro tipo) sem alocar memória adicional, diferentemente do `Substring`. Ele aloca na *Stack* apenas o endereço inicial e o *offset* da *string* na *Heap*. Se o *ReadOnlySpan* for convertido para `ToString()`, haverá alocação na *Heap*.
- **Classes *Record* e *Structs***: Classes são para objetos complexos e mutáveis; *structs* (que não suportam herança) são ideais para modelar *value objects* ou DTOs, sendo preferencialmente imutáveis e leves. A comparação de *structs* é sempre por valor.
- **Apresentação do Conceito de Record:**: Bruno Joaquim explicou que o `record` é um recurso que não existia no início do C\# e é alocado na Heap, sendo essencialmente um "açúcar sintático" para uma classe. Por padrão, o `record` é imutável, o que impede a alteração de suas propriedades após a criação, embora compare valor por valor (característica semelhante à `struct`), não por referência. É recomendado o uso de `record` para *requests*, *responses*, DTOs e *commands*.
- **Visualização da Compilação e Imutabilidade do Record:**: A ferramenta Sharplab foi utilizada para demonstrar que o compilador C\# trata o `record` como uma classe, gerando atributos específicos e utilizando o tipo `init` em vez de `set` para as propriedades. Isso garante que a definição dos valores seja permitida apenas no momento da criação do `record`, reforçando sua imutabilidade. A criação de um novo `record` a partir de um existente pode ser feita com a sintaxe `with`.
- **Funcionalidades Embutidas e Comparação de Valor:**: O `record` implementa automaticamente a comparação de valor (método `equals`), diferentemente de uma classe tradicional. Além disso, ele possui uma funcionalidade *out of box* de `ToString`, que concatena todas as propriedades e seus valores, facilitando a depuração. Victor Prospero resumiu que o `record` é um atalho de código (*syntax sugar*) para uma classe que aplica práticas como a comparação de valor (no `equals`) e a imutabilidade.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Utilização de Record Struct e Imutabilidade Preservada:**: Bruno Joaquim apresentou a alternativa `record struct`, que permite armazenar o `record` na *Stack* em vez da *Heap*. Contudo, a simples declaração de `record struct` remove a imutabilidade, expondo o método `set`. Para garantir a imutabilidade, mantendo o armazenamento na *Stack* e os benefícios do `record`, deve-se utilizar a declaração `read only record struct`.
- **Record e Mapeamento com Entity Framework:**: Gustavo Ferraz Fontes questionou sobre a compatibilidade e eventuais dificuldades no mapeamento de `record` ou `struct` com o Entity Framework. Bruno Joaquim explicou que, embora versões mais antigas do Framework pudessem apresentar problemas, exigindo classes em vez de `struct` ou `record`, as versões mais atuais oferecem suporte completo para o mapeamento desses tipos, geralmente tratando *Value Objects* como colunas na tabela principal.
- **Uso de Struct com Memory Cache:**: Wallace Silva perguntou se seria possível usar `struct` em um *Memory Cache* ao invés de classes para guardar dados em lote. Bruno Joaquim esclareceu que, embora o *Memory Cache* aloque dados na *Heap*, a alocação inicial dos objetos antes de serem enviados ao *cache* ocorreria na *Stack* se fosse utilizada a `struct`. Dessa forma, haveria um ganho de performance na alocação inicial dos objetos, pois o *Garbage Collector* não atuaria na região de memória do *Memory Cache*.
- **Alocação de Memória e Listas vs. Arrays:**: Bruno Joaquim iniciou a discussão sobre listas, afirmando que *arrays* são imutáveis e possuem um tamanho pré-determinado, com a memória alocada logo na criação. A `List` possui um tamanho dinâmico, mas internamente ela é baseada em um *array*. Quando o tamanho de uma `List` é excedido, um novo *array* interno com o dobro do tamanho é criado, processo que ocorre por baixo dos panos.
- **Otimização do Tamanho da Lista e Sparse Arrays:**: O padrão inicial de uma `List` é de quatro posições, e exceder essa capacidade leva à duplicação do *array* interno. O aumento de tamanho subsequente sempre segue uma potência de dois, como 4, 8, 16. A criação de muitos *arrays* esparsos (com posições não utilizadas) devido ao redimensionamento é ineficiente. Recomenda-se criar a lista com o tamanho correto se ele for conhecido antecipadamente, o que evita a alocação de múltiplos *arrays* e melhora a eficiência. Igor Romulo confirmou que o melhor seria inicializar a lista com o tamanho máximo esperado para otimizar o uso de memória.
- **Boas Práticas e Tipos de Coleções para o Retorno de Métodos:**: Bruno Joaquim aconselhou a usar sempre a propriedade `Count` ou `Length` em vez do método LINQ `Count()` ao trabalhar com listas, pois o último conta todos

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

os elementos. Ele também sugeriu evitar retornar ou receber `List` diretamente em métodos para prevenir a modificação da lista em *runtime*. O uso de `IEnumerable` ou `IReadOnlyCollection` é preferível para manter a imutabilidade e proteger a coleção contra alterações indesejadas.

- **Collections Baseadas em Hashtable (Hashset e Dictionary):**: Há uma categoria de coleções, como `Hashset` e `Dictionary`, que utilizam internamente uma *hashtable*. O `Hashset` é ideal para garantir a unicidade de elementos e é eficiente para inserções e buscas rápidas. Para operar, essas estruturas usam o método `GetHashCode` da classe para gerar um *hash*, que serve como indexador para acessar o valor em tempo constante.
- **Comparação de Performance entre Listas e Hashset:**: Um *benchmark* simples demonstrou que o tempo de acesso em uma `List` aumenta conforme o tamanho da coleção cresce, pois no pior caso é necessário percorrer a lista inteira. Em contraste, o `Hashset` não apresentou variação de tempo de acesso, independentemente do tamanho, devido ao uso do código *hash* para localização imediata do valor. Estruturas como `Hashset` e `Dictionary` são consideradas as melhores para busca, inserção e remoção, devido ao tempo constante de operação.
- **Demonstração do Garbage Collector (GC) e Alocação em Memória:**: Bruno Joaquim realizou uma demonstração de como diferentes padrões de leitura de arquivos afetam o *Garbage Collector*. A versão que carrega o arquivo inteiro para a memória (`load test 1`) resultou em grande alocação na *Large Object Heap* e na *Generation 0*, forçando o *Full GC* a rodar por mais tempo e consumindo 12% do tempo de execução. A versão otimizada que lê linha por linha (`load test 2`) demonstrou menor pressão no *GC* e menor alocação nessas regiões, mostrando a diferença entre alocar um objeto grande de uma vez e alocar progressivamente.

Próximas etapas sugeridas

- [ ] [Fabiano Felgas] Ver Gravação: Assistir gravação aula anterior sobre Dot Memory para entender profiling de memória.
- [ ] [Bruno Joaquim] Compartilhar Artigo: Deixar recomendação de leitura sobre W Collection Interface Tools.
- [ ] [O grupo] Upload Slides: Fazer upload da apresentação no portal.

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Revise as anotações do Gemini para checar se estão corretas. [Confira dicas e saiba como o Gemini faz anotações]

Envie feedback sobre o uso do Gemini para criar notas [breve pesquisa.]

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com